Retro Snake Game Using FPGA

Gunja **Jaswanth** (2310110476)

Ravuri Leela Venkata Sai **Krishna** (2310110576)

Dasari Yashaswi **Praneeth** Reddy (2310110456)

Pranav Krishna Palanisamy (2310110556)

Project Overview

What are we building?

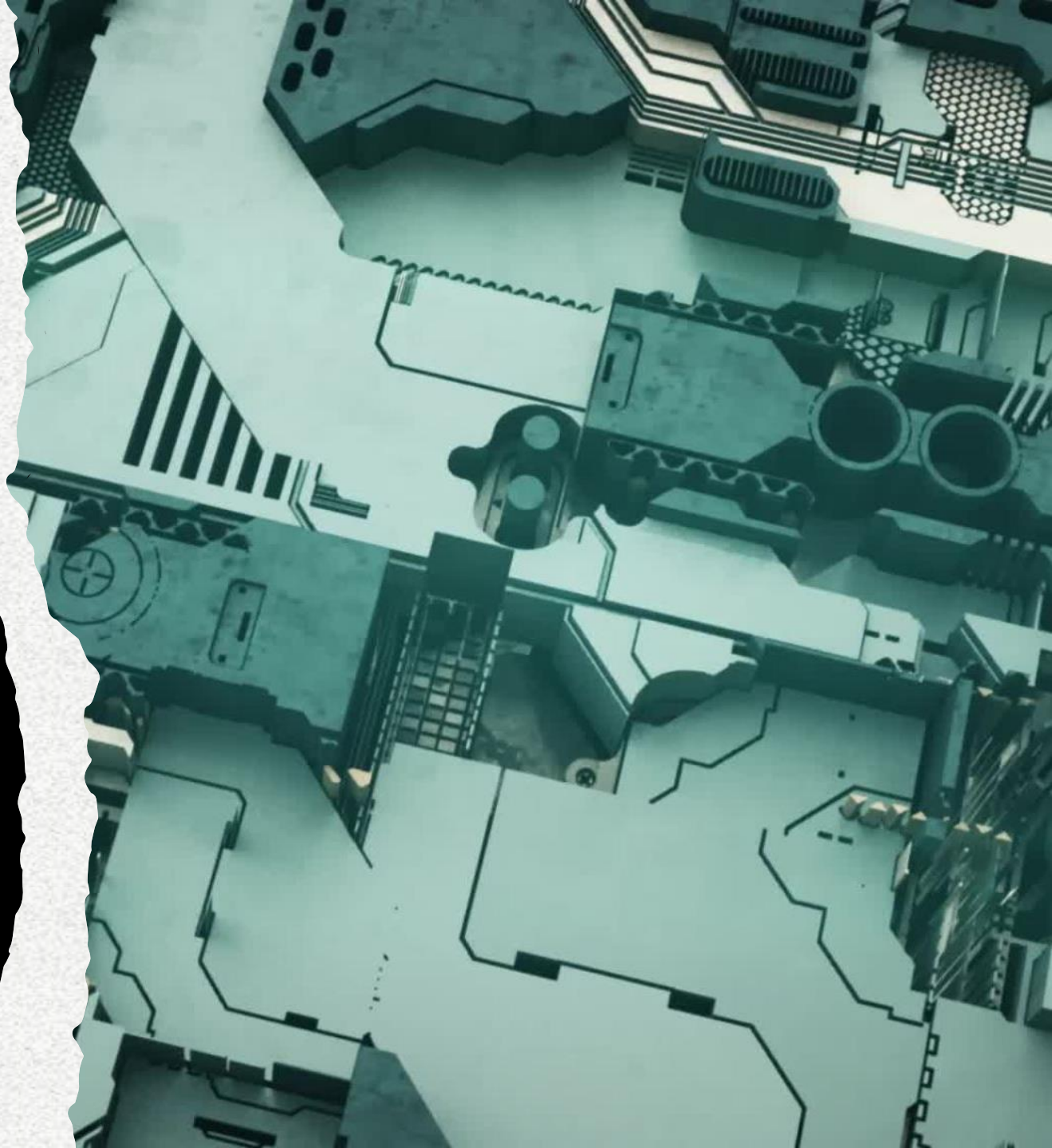
- A **real-time Snake Game implemented entirely on FPGA hardware**
- No processor, no software — **pure Verilog-based digital system**
- Displayed using **VGA (640×480 resolution)**
- Controlled using **physical buttons (UP, DOWN, LEFT, RIGHT)**

What makes this interesting?

- Game logic runs in **hardware → ultra-fast & parallel**
- Continuous **state updates using clock-driven design**
- Demonstrates how **simple logic → complete system**

Core Objectives

- Build a **fully functional interactive game on FPGA**
- Implement:
  - **Game Logic (movement, collision, growth)**
  - **Timing Control (clock division & update ticks)**
  - **Graphics Rendering (RGB + VGA sync)**



System Overview

- **Overall Working**

The project implements a **Snake Game entirely in FPGA hardware**, where all operations such as movement, collision detection, and rendering are handled using **parallel digital logic**. No processor is used, and the system works in real time using clock-driven execution.

- **Input → Processing → Output**

User inputs from buttons determine the snake's direction. The game logic updates the snake position, checks for collisions, and manages apple interaction. The final game state is converted into **VGA signals (RGB + sync)** and displayed continuously on the monitor.

- **Key Idea**

The system functions as a **synchronized hardware pipeline**, where all modules operate simultaneously to ensure smooth gameplay and real-time display.

Key Modules

Snake Logic Module

This is the core module of the system that controls the entire game behavior. It maintains the snake body using coordinate arrays, updates its position based on direction input, handles growth when the apple is eaten, and detects both border and self-collisions.

VGA Generator Module

Responsible for generating pixel coordinates and synchronization signals (horizontal and vertical sync). It ensures that each pixel is displayed at the correct time, enabling stable and flicker-free graphics on the monitor.

Clock Divider Module

This module converts the high-frequency FPGA clock into a suitable **VGA clock**, which is required for proper screen scanning and display timing.

Update Clock Module

Generates a slower clock signal that controls how often the snake moves. This effectively sets the **speed of the game**, ensuring the snake does not move too fast.

Input & Random Modules

The input module processes button presses and converts them into valid direction signals while preventing invalid moves. The random module generates pseudo-random positions for the apple, ensuring dynamic and unpredictable gameplay.

```
// ================= SNAKE DATA STRUCTURE =================

// Maximum number of snake segments
localparam MAX_LENGTH = 70;

// Arrays to store X and Y coordinates of each snake segment
// Index 0 → Head of the snake
// Remaining indices → Body segments
reg [9:0] snakeX [0:MAX_LENGTH-1];
reg [8:0] snakeY [0:MAX_LENGTH-1];

// Current length of the snake (initially 5 segments)
reg [6:0] snake_length = 5;
```

```
// ================ INITIALIZATION LOGIC ================

// Initialize snake position at the center of the screen
// Head starts at (320, 240)
// Each next segment is placed to the left of previous one

integer i;

always @(posedge clk) begin
    if (reset) begin
        for(i = 0; i < MAX_LENGTH; i = i + 1) begin
            snakeX[i] <= 320 - (i * CELL_SIZE);  // X decreases → horizontal line
            snakeY[i] <= 240;                    // Same Y → straight line
        end
    end
end
```

Snake Data Structure

Concept

- The snake is implemented using arrays that store coordinates of each segment.Each index represents one part of the snake, allowing easy tracking of position and movement.
- snakeX[ ] → Stores X-coordinates
- snakeY[ ] → Stores Y-coordinates

Key Features

- Supports up to 70 segments (maximum length)
- Head is always at index 0
- Remaining indices store the body segments in order
- Enables dynamic growth and smooth movement

```
// ================= NEXT POSITION (MOVEMENT) =================

// Compute next head position based on direction
always @* begin
    nextSnakeX = snakeX[0];
    nextSnakeY = snakeY[0];

    case(direction)
        LEFT:  nextSnakeX = snakeX[0] - CELL_SIZE;
        RIGHT: nextSnakeX = snakeX[0] + CELL_SIZE;
        UP:    nextSnakeY = snakeY[0] - CELL_SIZE;
        DOWN:  nextSnakeY = snakeY[0] + CELL_SIZE;
    endcase
end
```

```
// ================= BORDER COLLISION =================

// Check if next position hits screen boundary
assign hitBorder =
    (nextSnakeX < BORDER_W) ||
    (nextSnakeX > SCREEN_W - BORDER_W - CELL_SIZE) ||
    (nextSnakeY < BORDER_W) ||
    (nextSnakeY > SCREEN_H - BORDER_W - CELL_SIZE);


// ================= SELF COLLISION =================

// Check if head collides with body
always @* begin
    hitSelf = 0;
    for (j = 1; j < MAX_LENGTH; j = j + 1) begin
        if (j < snake_length) begin
            if (nextSnakeX == snakeX[j] && nextSnakeY == snakeY[j])
                hitSelf = 1;
        end
    end
end
```

Movement Logic & Collision Detection

- **Snake Movement Logic**

The snake moves based on **direction inputs (LEFT, RIGHT, UP, DOWN)**. The **next head position** is calculated from its **current coordinates**, such as **increasing X for RIGHT** or **decreasing Y for UP**. Movement is updated only on the **`move_tick` signal**, which **controls the speed** and prevents the snake from moving too fast.

- **Collision Detection Logic**

Before updating the position, the system checks for **collisions**. A **border collision** occurs when the snake crosses **screen limits**, while a **self-collision** occurs when the **head overlaps with its body**. This is implemented by **comparing head coordinates** with all **body segments using a loop**.

Random Module

```
module Random(
    input VGA_clk,
    output reg [9:0] randX = 10'h155,
    output reg [8:0] randY = 9'h0E7
);

always @(posedge VGA_clk) begin
    randX <= {randX[8:0], randX[9]^randX[6]};
    randY <= {randY[7:0], randY[8]^randY[4]};
end

endmodule
```

- Why randX = 10'h155 and randY = 9'h0E7??

These are just some random values initialized, we can keep any values.

- Why is an XOR operation being done on the 9th & 6th in the X coordinate??

Random Assumption to generate random pattern

- **Random Module (LFSR)**

The random module generates pseudo-random values for apple placement using a **Linear Feedback Shift Register**. It updates every clock cycle and produces a changing sequence using shift and XOR operations.

- **Example (4-bit):**

`1011 → 0111 → 1111 → 1110 → …` [XOR of MSB and it's adjacent bit]

- **Apple Position Logic**

```
appleX = ((randX % N) * CELL_SIZE) + BORDER_W;
```

`randX % N` → limit inside screen

× `CELL_SIZE` → align to grid

+ `BORDER_W` → avoid walls

Apple Logic

- The apple is placed at **random positions** on the screen using a simple random generator.
- Its position is always kept **inside the borders** and **aligned to the grid**, so it matches the snake movement.
- When the snake's head touches the apple, the snake **grows**, and a **new apple appears at another random location**, keeping the game continuous.

```
// ================ APPLE POSITION ================

// Apple is placed within valid screen area and aligned to grid
always @(posedge clk) begin
    if (reset) begin
        appleX <= ((randX % ((SCREEN_W - 2*BORDER_W)/CELL_SIZE)) * CELL_SIZE) + BORDER_W;
        appleY <= ((randY % ((SCREEN_H - 2*BORDER_W)/CELL_SIZE)) * CELL_SIZE) + BORDER_W;
    end
end


// ================ EAT APPLE CONDITION ================

// Check overlap between snake head and apple
assign eatApple =
    (nextSnakeX < appleX + CELL_SIZE) &&
    (nextSnakeX + CELL_SIZE > appleX) &&
    (nextSnakeY < appleY + CELL_SIZE) &&
    (nextSnakeY + CELL_SIZE > appleY);
```

```
// ================ ON EATING APPLE ================

always @(posedge clk) begin
    if (eatApple) begin
        grow <= grow + 3;  // trigger growth

        // generate new apple position
        appleX <= ((randX % ((SCREEN_W - 2*BORDER_W)/CELL_SIZE)) * CELL_SIZE) + BORDER_W;
        appleY <= ((randY % ((SCREEN_H - 2*BORDER_W)/CELL_SIZE)) * CELL_SIZE) + BORDER_W;
    end
end
```

```
// SMOOTH GROWTH
if (grow > 0) begin
    snakeX[snake_length] <= snakeX[snake_length-1];
    snakeY[snake_length] <= snakeY[snake_length-1];
    snake_length <= snake_length + 1;
    grow <= grow - 1;
end

// EAT APPLE
if (eatApple) begin
    grow <= grow + 3;

    appleX <= ((randX % ((SCREEN_W - 2*BORDER_W)/CELL_SIZE)) * CELL_SIZE) + BORDER_W;
    appleY <= ((randY % ((SCREEN_H - 2*BORDER_W)/CELL_SIZE)) * CELL_SIZE) + BORDER_W;
end
```

```
always @(posedge VGA_clk) begin
    if (xCount == H_TOTAL-1) begin
        xCount <= 0;
        if (yCount == V_TOTAL-1) yCount <= 0;
        else yCount <= yCount + 1;
    end else xCount <= xCount + 1;

    displayArea <= (xCount < 640) && (yCount < 480);
    hSync <= (xCount >= 656 && xCount < 752);
    vSync <= (yCount >= 490 && yCount < 492);
end

assign VGA_hSync = ~hSync;
assign VGA_vSync = ~vSync;
```

Growth Mechanism & VGA System

Snake Growth Mechanism

The snake grows using a **counter-based approach**. When it eats an apple, a growth counter increases. During each update cycle, one segment is added at the tail while the counter decreases, resulting in smooth and gradual growth.

VGA Display System

- This block continuously scans the screen from left to right and top to bottom, generating pixel positions and sync signals so the VGA display can correctly show the image.
- These pixel positions are then used by the game logic to decide what color (snake, apple, or background) should be displayed at each point on the screen.

Clock Design

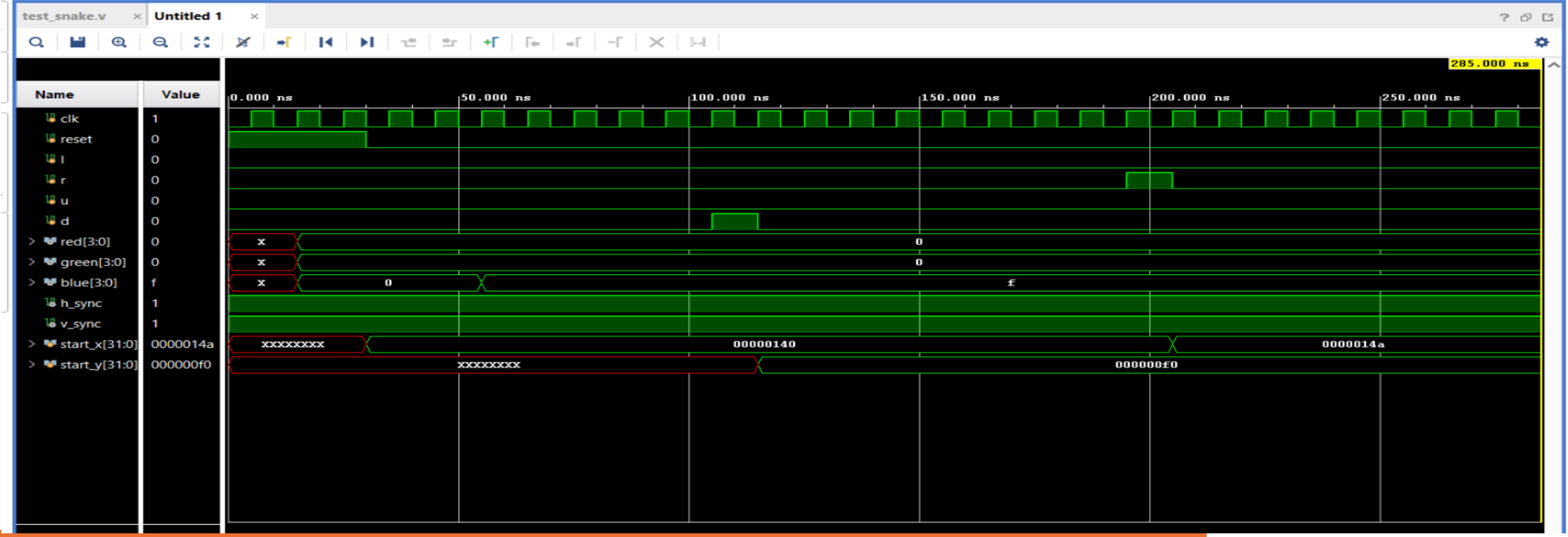
- The clock system is designed to generate **different timing signals** required for the game. A **clock divider** is used to derive a **suitable frequency for VGA display**, while a separate **update clock** controls the speed of **snake movement**.

- To ensure controlled updates, an **edge-detection mechanism** is used to generate a move signal only at specific clock transitions. This allows synchronized movement, stable display output, and smooth gameplay.

```
// ================= CLOCK DIVIDER =================
module ClockDivider(input clk, output VGA_clk);
    reg [1:0] div_cnt = 0;
    always @(posedge clk) div_cnt <= div_cnt + 1;
    assign VGA_clk = div_cnt[1];
endmodule

// ================= UPDATE CLOCK =================
module UpdateClock #(parameter COUNT_MAX = 3000000)(
    input clk,
    output reg update_clk = 0
);
    reg [22:0] count = 0;

    always @(posedge clk) begin
        if (count == COUNT_MAX) begin
            count <= 0;
            update_clk <= ~update_clk;
        end else count <= count + 1;
    end
endmodule
```

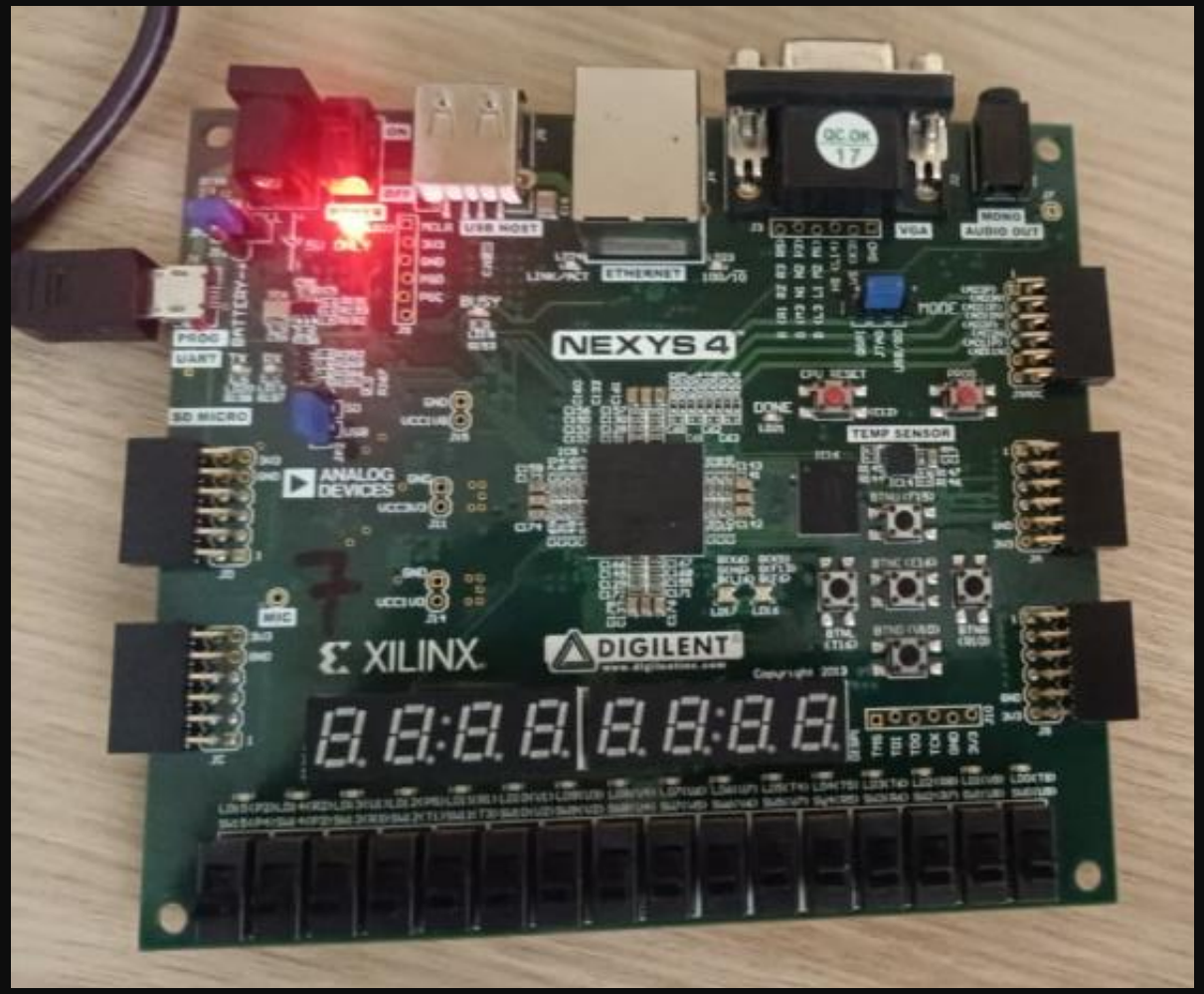


Testing & Verification

- A dedicated testbench was developed to simulate the design and verify functionality before hardware implementation. The testing focused on ensuring correct movement behavior, proper direction changes, and accurate position updates of the snake.
- Directional validation was performed by observing coordinate changes, such as X increasing for RIGHT movement and Y increasing for DOWN movement. This process helped identify and fix issues early, ensuring reliable and correct system behavior before deployment on FPGA.

Hardware Implementation

- The design was successfully implemented on an FPGA board, enabling real-time execution of the Snake game. The VGA interface was used to display the game by connecting RGB and synchronization signals to the monitor.
- User inputs were provided through push buttons for controlling direction, while a reset button was used to restart the game. All connections were defined using a constraints (XDC) file, ensuring correct mapping between FPGA pins and external components.



Challenges and Solutions

Challenges:	Clock synchronization issues → irregular movement	Snake growth glitches → abrupt expansion	Collision detection → self checks
Solutions:	Edge-based movement for proper timing control	Counter-based gradual growth for smooth expansion	Modular debugging to isolate and fix issues

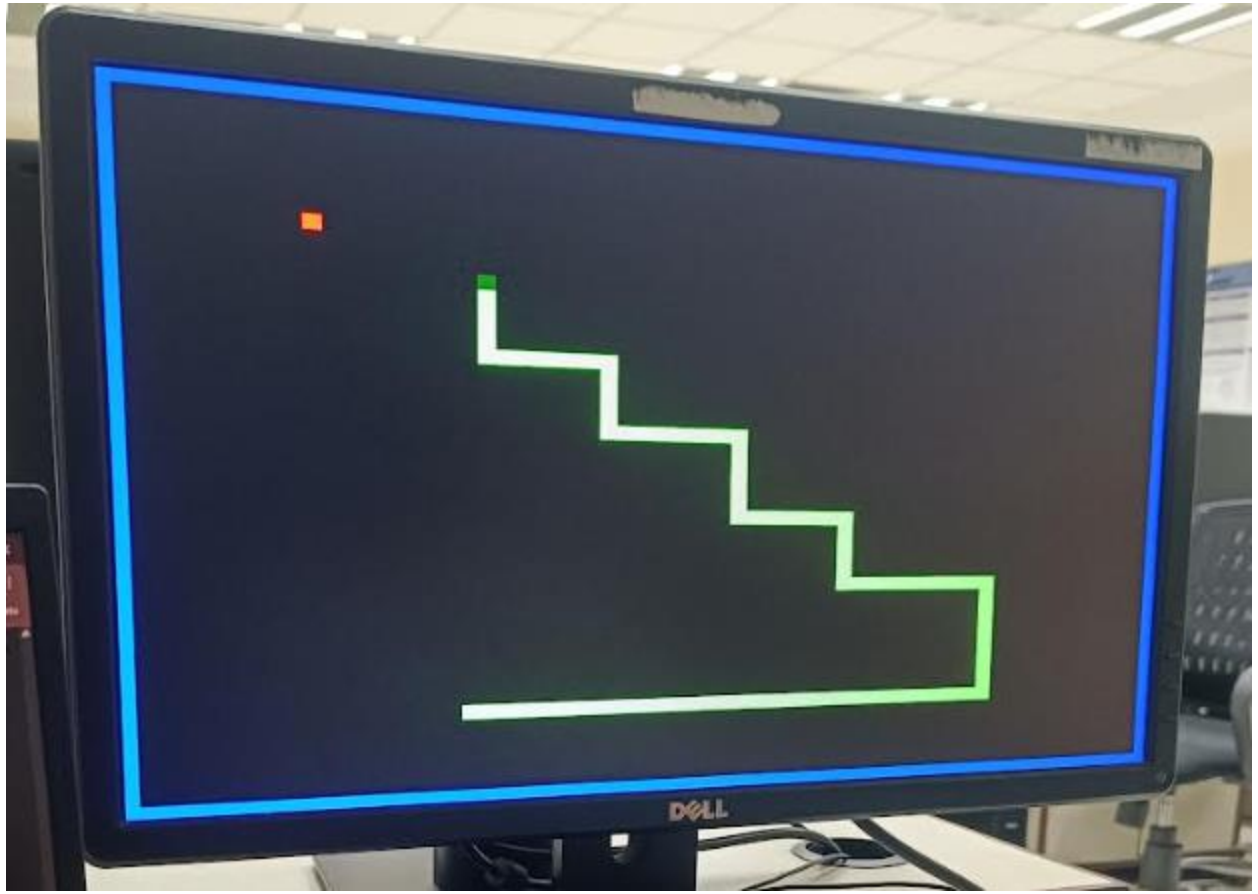
Results

The implemented Snake game on FPGA demonstrates smooth and consistent movement controlled by clock-based updates.

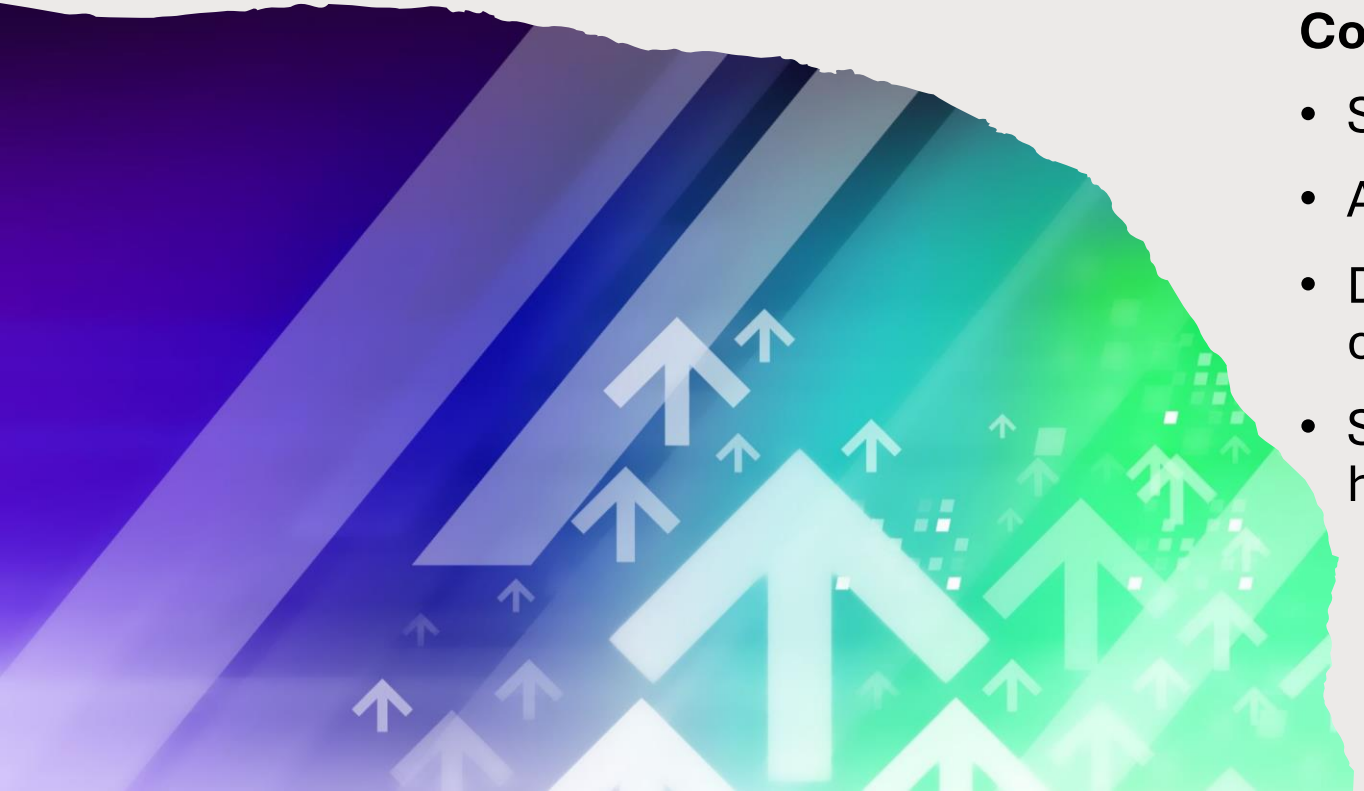
The system accurately detects **collisions with walls and the snake's own body**, ensuring correct game behavior.

The VGA module successfully generates a real-time graphical output at **640×480** resolution, displaying the snake, food, and boundaries with proper synchronization.

Overall, the design achieves responsive gameplay and stable visual performance, validating the effectiveness of hardware-based implementation.



Future Improvements and Conclusion



Future Improvements

- Add score display and levels
- Increase game speed dynamically
- Improve graphics (colors, animations)
- Add sound effects using audio module
- Implement pause functionality

Conclusion

- Successfully implemented Snake Game on FPGA
- Achieved real-time VGA-based display
- Demonstrated smooth movement and accurate collision detection
- Showcased concepts of digital design and hardware-based graphics

Thank You